

Exercício 01

riddle(X, _) :- length(X, XL), XL = 0.

riddle(_, Y) :- length(Y, YL), YL = 0.

Verifica se pelo menos uma das listas fornecidas (X ou Y) é vazia.

Reimplementação mais eficiente:

riddle([], _).

riddle(_, []).

Exercício 02

mystery(Item, List) :- append([Item], List).

Insere um elemento no início de uma lista.

Reimplementação sem append:

mystery(Item, List, [Item|List]).

Exercício 03

puzzle(_, [], []).

puzzle(E, [_|Tail], Result) :-

 puzzle(E, Tail, TailResult),

 append(TailResult, [E], Result).

Adiciona o elemento E ao final da lista Result gerada a partir de [_|Tail].

Reimplementação sem append:

puzzle(_, [], []).

puzzle(E, [_|Tail], Result) :-

 puzzle(E, Tail, TailResult),

 Result = TailResult ++ [E].

Exercício 04

fun dividir nil = (nil, nil)

 | dividir [a] = ([a], nil)

 | dividir (h1::h2::t) =

 let

 val (esq, dir) = dividir t

 in

 (h1::esq, h2::dir)

 end;

Reescrita com recursão de cauda:

fun dividir list =

 let

 fun aux(nil, esq, dir) = (rev esq, rev dir)

 | aux([a], esq, dir) = (rev (a::esq), rev dir)

 | aux(h1::h2::t, esq, dir) = aux(t, h1::esq, h2::dir)

 in

 aux(list, nil, nil)

 end;

Exercício 05

fun prefixCopy x 0 = x

 | prefixCopy nil _ = nil

```
| prefixCopy (head::tail) n = head :: (prefixCopy tail (n-1));
```

Reescrita com recursão de cauda:

```
fun prefixCopy x n =  
  let  
    fun aux(nil, _, acc) = rev acc  
      | aux(_, 0, acc) = rev acc  
      | aux(head::tail, n, acc) = aux(tail, n-1, head::acc)  
  in  
    aux(x, n, nil)  
  end;
```

Exercício 06

a) Preferência por grandfather2:

Se há muitos fatos parent e poucos fatos male, verificar male primeiro reduz o número de combinações possíveis mais cedo, economizando tempo.

b) Preferência por grandfather1:

Se há muitos fatos male e poucos fatos parent, verificar parent primeiro reduz o número de combinações possíveis mais cedo.

c) Ordem de fatos:

Para evitar backtracking em grandfather1:

Ordem: male(joe). male(bob). parent(joe, bob). parent(bob, alice).

Passos:

Prolog encontra primeiro parent(joe, bob).

Verifica que joe é male.

Encontra parent(bob, alice).

Para evitar backtracking em grandfather2:

Ordem: parent(joe, bob). parent(bob, alice). male(joe). male(bob).

Passos:

Prolog encontra male(joe).

Verifica parent(joe, bob) e parent(bob, alice).